

Final Lab Exam – Operating Systems

Dear Students,

This is your **final practical exam** in the Operating Systems course. Please read the tasks carefully and complete them during your lab session.

1. **Process Synchronization** Explain what we mean by process synchronization in Operating Systems. Why is it important when multiple processes run at the same time?
2. **Critical Section & Race Condition** Discuss the situations that require synchronization. Focus especially on the concepts of *Critical Section* and *Race Condition*.
3. **Readers–Writers Problem** Describe the Readers–Writers Problem. Why is it important, and what are the possible solutions?
4. **Dining Philosophers Problem** Talk about the Dining Philosophers Problem. Explain why it is considered a classic example of deadlock and resource sharing, and mention strategies to avoid deadlock.
5. **Implementation Task** Choose one of the problems above (Readers–Writers or Dining Philosophers).
 - Write a program in **C++ or Python** that solves it.
 - Run your program on your Lab PC and show the output.

Q1/**Process Synchronization** Explain what we mean by process synchronization in Operating Systems. Why is it important when multiple processes run at the same time?

Process synchronization is defined as "a mechanism in operating systems used to manage the execution of multiple processes that access shared resources". Its fundamental purpose is to coordinate **cooperating processes**—those that "can affect or be affected by other processes executing in the system"—to ensure that shared data remains consistent and reliable.

The Importance of Synchronization

When multiple processes or threads run concurrently or in parallel, they often share a logical address space or common data. Without synchronization, several critical issues can arise:

- **Race Conditions:** This is a situation where "the result of multiple process/thread execution in the critical section differs according to the order in which the threads execute". Because processes can be interrupted at any point in their instruction stream, the final value of shared data depends on the specific order of interleaved execution, which often leads to corrupted or incorrect values.
- **Data Inconsistency:** Concurrent access without coordination can lead to "conflicting changes, where one process's update is overwritten by another, causing the data to become unreliable and incorrect".
- **Deadlock:** Improper synchronization may cause two or more processes to get "stuck, each waiting for the other to release a resource," preventing any of them from continuing.
- **Loss of Data:** This occurs when multiple processes attempt to write to the same resource simultaneously; if one overwrites the data before another finishes, "important information can be lost".

Q2/**Critical Section & Race Condition** Discuss the situations that require synchronization. Focus especially on the concepts of Critical Section and Race Condition.

1. The Critical Section

A **critical section** is defined as a specific segment of code "in which the process may be accessing—and updating—data that is shared with at least one other process".

- **The Problem:** The core challenge, known as the **critical-section problem**, is to design a protocol where "when one process is executing in its critical section, no other process is allowed to execute in its critical section".
- **Requirements:** Any solution must ensure **mutual exclusion**, meaning only one process at a time can be active in its critical section; **progress**, ensuring decisions on who enters next aren't postponed indefinitely; and **bounded waiting**, which limits how long a process must wait to enter.

2. Race Conditions

A **race condition** occurs when "several processes access and manipulate the same data concurrently and the final outcome of the execution depends on the particular order in which the access takes place".

- **Mechanism:** These situations often arise because high-level code (like `count++`) is translated into multiple low-level machine instructions (load, increment, store). If a process is interrupted between these steps, an interleaved execution sequence can result in **data inconsistency** or **loss of data**.
- **Example:** If two processes attempt to update a shared counter simultaneously, one's update might be overwritten by the other, leading to an "unreliable and incorrect" final value.

Q3/Readers–Writers Problem Describe the Readers–Writers Problem. Why is it important, and what are the possible solutions?

The **Readers–Writers Problem** is defined as a "classic synchronization issue in operating systems" that involves "coordinating access to shared data (e.g., database, file) by multiple processes or threads". It identifies two types of users: **Readers**, who can access data "simultaneously without causing inconsistency," and **Writers**, who require "exclusive access" because they modify the data.

Importance of the Problem

The problem is significant because it "highlights the need for proper synchronization when multiple processes access shared data". Its primary goals are:

- **Ensuring Data Integrity:** It prevents "data corruption" by ensuring that "no readers are allowed while writing" and only "one writer can access the data at a time".
- **Efficiency:** It allows "multiple readers [to] access data together if no writer is writing," which "improves read efficiency".
- **Defining Access Rules:** It establishes which operations are allowed concurrently:
 - **Reading & Reading:** Allowed.
 - **Writing & Writing:** Not Allowed.
 - **Writing & Reading:** Not Allowed.

Possible Solutions and Variants

The sources outline different strategies to manage these processes, primarily focusing on priority and fairness:

1. **Readers Preference:** Readers are given priority so that "no reader waits if the resource is available for reading, even if a writer is waiting". While this maximizes concurrent reading, "writers may suffer from starvation" if readers arrive continuously.
2. **Writers Preference:** Writers are prioritized to ensure they "won't starve," though this may result in readers "waiting longer".
3. **Fair/Balanced Solutions:** To avoid starvation for either group, systems can use "queues or advanced semaphores" to ensure "no starvation and balanced access".

Q4/Dining Philosophers Problem Talk about the Dining Philosophers Problem. Explain why it is considered a classic example of deadlock and resource sharing, and mention strategies to avoid deadlock.

The **Dining Philosophers Problem** is a "classic synchronization and concurrency problem that deals with resource sharing, deadlock, and starvation in systems where multiple processes require limited resources". It involves 'n' philosophers sitting around a circular table who alternate between "thinking and eating". To eat, a philosopher "needs two chopsticks, one on their left and one on their right," but the number of chopsticks is equal to the number of philosophers, meaning each chopstick is "shared between two neighboring philosophers".

Deadlock and Resource Sharing

This problem is a foundational example of **resource sharing** because each process (philosopher) competes for a limited set of resources (forks) that are shared with its immediate neighbors. It is considered a classic example of **deadlock** because of a specific interleaving scenario: if every philosopher picks up their "left forks before any of them tries to pick up a right fork," the system enters a state where "all the Philosophers are waiting for the right fork but no one will execute a single instruction".

Strategies to Avoid Deadlock

The sources describe several protocols to resolve these challenges:

- **Limiting Capacity:** One approach is to "limit the number of philosophers entering the room to four" (in a 5-fork system). This ensures that "one of the philosophers will eventually get both the fork[s]" and proceed, preventing a total system stall.
- **Asymmetric Solution (Chandy/Misra):** In this protocol, the first four philosophers pick up their left fork then their right, but the "fifth philosopher waits for the right fork and then for the left fork". This breaks the circular wait condition and is "Free from Deadlock".
- **Even/Odd Strategy:** As established in our conversation, a valid asymmetric variation is to have **even-indexed philosophers** pick up their left fork first, and
- **Monitors:** A "Monitor maintains an array" of available forks and only allows a philosopher to eat if both neighboring forks are free. The **takeForks** operation waits on a **condition variable** until this state is met.

